Cloud-Based Linux Server Deployment with GitHub Version Control and Automation

- **Student Name**: Aishwarya Mayekar
- **Batch Code**: AUG26-A530
- **Institute**: IT Vedant
- **Course**: DevOps with AWS
- **Project Type**: DevOps Fundamentals Mini Project
- **Submission Date**: 24 October 2026

Project Overview

This project demonstrates the deployment of a basic website on an Ubuntu Linux server hosted on Amazon Web Services (AWS). The server runs on an Amazon EC2 instance, and Apache is used to serve the website to users through a web browser.

A custom Virtual Private Cloud (VPC), public subnet, Internet Gateway, route table, and Security Group were configured to provide network connectivity and control access to the server.

Git and GitHub were used for version control and project storage. Three Bash scripts were developed to check server health, monitor the Apache service, and create timestamped website backups. System logs were also reviewed to understand authentication activity, SSH events, errors, and warnings.

Project objectives

1. Create a custom AWS network using a VPC and subnet.
2. Launch and access an Ubuntu EC2 instance.
3. Install and configure Apache.
4. Deploy a webpage and access it through the public IP.
5. Manage project files using Git and GitHub.
6. Automate server checks and backup tasks using Bash.
7. Review Linux logs for troubleshooting and reporting.

AWS Architecture

- **VPC:** Logically isolated AWS network.
- **Public subnet:** The subnet in which you launched the server.
- **Internet Gateway:** Connects the VPC to the internet.
- **Route table:** Contains the route to the Internet Gateway, typically 0.0.0.0/0.
- **Security Group:** Controls inbound and outbound traffic to the EC2 instance.
- **EC2:** The virtual server running Ubuntu.
- **Apache:** The web server that serves your HTML file.

Installation Commands

Command	Explanation
`sudo apt update`	Refreshes Ubuntu's package index.
`sudo apt upgrade -y`	Installs available package upgrades.
`sudo apt install git apache2 curl vim -y`	Installs the required tools.
`sudo systemctl enable --now apache2`	Starts Apache and enables it on boot.
`sudo systemctl status apache2`	Checks Apache's service status.
`curl http://localhost`	Tests the web server locally.
`git init`	Initializes a Git repository.
`git add .`	Stages changes for a commit.
`git commit -m "message"`	Saves a snapshot of staged changes.
`git push`	Pushes committed changes to the remote repository.

Bash Scripts

Server Health Check — health-check.sh

It displays CPU information, memory usage, disk usage, logged-in users, running processes, and server uptime. It helps an administrator identify possible resource problems.

Apache Service Monitor — apache-monitor.sh

It checks whether Apache is active. If the service is stopped, the script attempts to restart it and verifies whether the restart succeeded. The script reports success or failure.

Website Backup — backup.sh

It compresses files from /var/www/html, adds a timestamp to the archive name, and saves the archive in the project's Backup directory. Timestamped files help preserve multiple backups.

Log Analysis

- **sudo journalctl -u ssh --since "7 days ago" --no-pager** — Examines SSH service logs.
- **sudo journalctl -p err --since "7 days ago" --no-pager** — Displays error-priority journal entries.
- **sudo journalctl -p warning --since "7 days ago" --no-pager** — Displays warning-priority entries.

Document challenges and solutions

1. Git email configuration

Problem: Git could not create a commit because the author email was not configured.

Solution: Configured the Git username and email using git config --global, then created the commit and pushed it to GitHub.

2. Missing Git remote

Problem: Git returned No such remote 'origin' when trying to update the repository URL.

Solution: Added the remote using git remote add origin, verified it using git remote -v, and pushed the project.

3. Apache webpage issue

Problem: Apache's service unit was not found, and the webpage was initially inaccessible.

Solution: Checked the website directory, installed Apache if necessary, started the service, and verified the webpage locally and through the EC2 public IP.

Learning outcomes and conclusion

- Creating and configuring AWS networking resources.
- Launching and accessing an Ubuntu EC2 instance.
- Installing and managing Apache on Linux.
- Hosting a website on a cloud server.

- Using Git and GitHub for version control.
- Writing Bash scripts for server monitoring, service management, and backups.
- Reviewing system logs for troubleshooting.
- Understanding the importance of access control and cloud cost management.

Conclusion

This project provided practical exposure to cloud infrastructure deployment, Linux server administration, website hosting, version control, and Bash automation. It demonstrated how AWS and open-source tools can be combined to deploy and manage a simple web application. The project also improved my understanding of troubleshooting, system monitoring, and maintaining project documentation.